

DoomOS Master Implementation Prompts (Hardened & Battle-Tested)

Production-Ready Prompts for Antigravity

Target Host: macOS (Apple Silicon arm64) & Linux hosts
Target Guest OS: DoomOS (x86_64, KDE Plasma 6 on Wayland, Btrfs, Calamares)
Status: Audited, Hardened, and Verified for Antigravity Execution

Pre-Flight Instructions for Antigravity

- **Execution Order:** Execute strictly one phase at a time.
 - **Docker Prerequisite:** Ensure Docker Desktop is running before launching Phase 1.
 - **Recommended Slash Commands:**
 - Use `/plan` for **Phase 1** and **Phase 5**.
 - Use `/goal` for **Phase 4** and **Phase 6** to allow autonomous script authoring and compilation.
-

Phase 1: Build Infrastructure & Docker Scaffolding (Hardened)

Antigravity Slash Command: `/plan`
Key Hardening: Adds `--platform linux/amd64` and `--privileged` flags to guarantee a native x86_64 Linux build environment on Apple Silicon Macs with full loop-device support.

Prompt 1

Act as a Senior Linux Systems Engineer. We are constructing "DoomOS", an Arch-based Linux distr

Task: Initialize the project directory structure and create the containerized build engine.

Requirements:

1. Create the complete directory tree inside the project workspace:

```
doomos-builder/  
├── Dockerfile  
├── build.sh  
├── profile/  
│   ├── packages.x86_64  
│   ├── pacman.conf  
│   └── airootfs/  
│       ├── etc/  
│       └── usr/  
└── output/
```

2. Write a production-grade Dockerfile that:
 - Uses the base image 'FROM --platform=linux/amd64 archlinux:base-devel'.
 - Installs: archiso, git, squashfs-tools, dosfstools, libisoburn, xorriso, grub, and mtools.
 - Sets the working directory to '/build' and entrypoint to '/build/build.sh'.
3. Write 'build.sh' with bash strict mode (set -euo pipefail):
 - Validates that the script is running inside an x86_64 Linux container with root/privileged
 - Includes progress banners, directory cleanups, and validation checks.
4. Write a host launcher script 'run-builder.sh' for macOS/Linux:
 - Validates that Docker is running.
 - Builds the Docker image 'doomos-builder:latest' with '--platform linux/amd64'.
 - Runs the container with '--privileged --platform linux/amd64 -v "\$PWD":/build'.
5. Verify all files are created and permissions set (+x on scripts).

Phase 2: Package Manifest & Core Rootfs Configuration (Hardened)

Key Hardening: Explicitly bundles `plymouth` to prevent `mkinitcpio` hook failure, enables `[multilib]` repositories, and configures the early-KMS initramfs pipeline.

Prompt 2

Act as a Senior Systems Engineer on the DoomOS project.

Task: Construct the core package manifest and base filesystem configuration in accordance with

Requirements:

1. Create 'profile/packages.x86_64' with a curated, deduplicated manifest covering:
 - Dual Kernels: linux-zen, linux-zen-headers, linux-lts, linux-lts-headers, linux-firmware,
 - Boot & Hardware: plymouth, btrfs-progs, zram-generator, sudo, polkit, dbus, networkmanager
 - 32-bit Multi-lib Runtime: lib32-mesa, lib32-vulkan-radeon, lib32-vulkan-intel, lib32-nvidia
2. Create 'profile/pacman.conf' with:
 - Enabled [core], [extra], and [multilib] repositories with official Arch mirror URLs.
 - ParallelDownloads = 5.
 - Color, CheckSpace, and ILoveCandy options enabled.
3. Create 'profile/airootfs/etc/os-release' identifying the OS:
NAME="DoomOS"
PRETTY_NAME="DoomOS Linux (KDE Plasma Edition)"
ID=doomos
ID_LIKE=arch
HOME_URL="https://doomos.org"
4. Create 'profile/airootfs/etc/mkinitcpio.conf' with the hardened hook sequence:
HOOKS=(base udev plymouth autodetect modconf kms keyboard keymap consolefont block btrfs fil

Phase 3: Display Stack, SDDM & KDE Plasma 6 Customization (Hardened)

Key Hardening: Removes deprecated `plasma-wayland-session` (absorbed into `plasma-workspace` in Plasma 6 on Arch), installs `plasma-meta` and Wayland driver stack, and configures zero-latency VRR.

Prompt 3

Act as a Senior Desktop Systems Engineer on DoomOS.

Task: Configure the Wayland display stack, SDDM display manager, and KDE Plasma 6 desktop environment.

Requirements:

- Append the desktop suite to 'profile/packages.x86_64':
 - Graphics: mesa, vulkan-radeon, vulkan-intel, nvidia-dkms, nvidia-utils, xwayland.
 - Audio: pipewire, wireplumber, pipewire-pulse, pipewire-alsa, pipewire-jack.
 - Desktop (Plasma 6): plasma-meta, sddm, sddm-kcm, dolphin, konsole, kate, plasma-systemmonitor (NOTE: Do NOT include 'plasma-wayland-session'; Plasma 6 natively runs Wayland via plasma-workspace).
- Configure SDDM Live Autologin in 'profile/airootfs/etc/sddm.conf.d/liveuser.conf':

```
[Autologin]
User=liveuser
Session=plasma
```
- Configure KWin Wayland parameters in 'profile/airootfs/etc/xdg/kwinrc':

```
[Wayland]
AllowTearingAtFullscreen=true
[Displays]
AdaptiveSync=Always
```
- Configure high-DPI scaling flags in 'profile/airootfs/etc/environment.d/10-hidpi.conf':

```
QT_QPA_PLATFORM="wayland;xcb"
ELECTRON_OZONE_PLATFORM_HINT="auto"
GDK_BACKEND="wayland,x11,*"
```
- Write a default shell profile in 'profile/airootfs/etc/skel/.bashrc' and 'profile/airootfs/etc/skel/.profile'.

Phase 4: Problem-Solving Subsystems Implementation (Hardened)

Antigravity Slash Command: `/goal`

Key Hardening: Provides complete code implementations for all 6 subsystems, verifies executable permissions (`chmod +x`), and configures PipeWire loopback audio.

Prompt 4

Act as a Principal Linux Kernel & Systems Architect.

Task: Implement all 6 problem-solving subsystems specified in Section 2 of DoomOS_Master_Specif

Subsystems to Implement:

1. NVIDIA DKMS Guardian:
 - Create '/usr/share/libalpm/hooks/90-doomos-nvidia-guardian.hook' inside airootfs, triggeri
 - Write '/usr/bin/doomos-nvidia-verify' (chmod +x) checking module presence via 'modinfo -k
2. Btrfs Snapper Auto-Rollback:
 - Add snapper, snap-pac, and grub-btrfs to 'profile/packages.x86_64'.
 - Create 'profile/airootfs/etc/snapper/configs/root' with TIMELINE_CREATE=yes, NUMBER_LIMIT=
 - Enable 'grub-btrfsd.service' via systemd symlinks inside airootfs.
3. Wayland Portal & Audio Loopback:
 - Add xdg-desktop-portal-kde and xdg-desktop-portal-gtk to packages manifest.
 - Create 'profile/airootfs/etc/pipewire/pipewire.conf.d/10-loopback-share.conf' configuring
4. Developer Sysctl Tuning & Rootless Containers:
 - Create 'profile/airootfs/etc/sysctl.d/99-developer-performance.conf':
fs.inotify.max_user_watches = 1048576
fs.inotify.max_user_instances = 2048
vm.max_map_count = 2147483642
net.core.somaxconn = 8192
 - Pre-populate '/etc/subuid' and '/etc/subgid' templates for user namespaces.
5. Laptop Battery & Sleep Policy:
 - Add auto-cpufreq and power-profiles-daemon to packages manifest.
 - Create 'profile/airootfs/etc/systemd/sleep.conf.d/10-doomos-power.conf' with MemorySleepMo
6. HDR & Game Launch Wrapper:
 - Write 'profile/airootfs/usr/bin/doom-game' (chmod +x) that checks display HDR capability v

Verify all script syntax and directory permissions upon completion.

Phase 5: Calamares Graphical Installer & Dual-Boot RTC (Hardened)

Antigravity Slash Command: /plan

Key Hardening: Provides both partition.conf AND mount.conf to guarantee proper creation of Btrfs @, @home, @snapshots, and @var_log subvolumes on user disks.

Prompt 5

Act as a Systems Integration Engineer on DoomOS.

Task: Configure the Calamares installer with custom DoomOS branding and automated Btrfs subvolumes.

Requirements:

1. Add 'calamares', 'calamares-extensions', and 'ckbcomp' to 'profile/packages.x86_64'.
2. Create 'profile/airootfs/etc/calamares/settings.conf':
 - Modules sequence: welcome, locale, keyboard, partition, users, unpackfs, doomos-rtc, machine-id
3. Create 'profile/airootfs/etc/calamares/modules/partition.conf':
 - Sets defaultFileSystemType: "btrfs".
 - Enables swap partition or swapfile configuration.
4. Create 'profile/airootfs/etc/calamares/modules/mount.conf':
 - Configures exact Btrfs subvolume mounts:
 - subvolume: "@", mountPoint: "/"
 - subvolume: "@home", mountPoint: "/home"
 - subvolume: "@snapshots", mountPoint: "/.snapshots"
 - subvolume: "@var_log", mountPoint: "/var/log"
5. Create 'profile/airootfs/etc/calamares/modules/doomos-rtc/main.py':
 - Inspects target disk EFI directories for '/EFI/Microsoft/Boot/bootmgfw.efi'.
 - If Windows is found, executes 'timedatectl set-local-rtc 1 --adjust-system-clock'.
6. Create 'profile/airootfs/etc/calamares/branding/doomos/branding.desc':
 - Product name: "DoomOS"
 - Version: "1.0.0 Rolling"
 - Visual styling: Deep slate background with cyan/emerald highlights.

Phase 6: Master ISO Packaging Pipeline (Hardened)

Antigravity Slash Command: `/goal`

Key Hardening: Uses `mkarchiso` native orchestration engine inside the container to build, compress (zstd level 19), generate signed EFI binaries, and master the hybrid ISO.

Prompt 6

Act as a Release & Build Engineer on DoomOS.

Task: Complete the build pipeline script to compile, compress, and master the bootable hybrid ISO.

Requirements:

1. Finalize 'build.sh' inside doomos-builder:
 - Executes 'mkarchiso -v -w /build/work -o /build/output /build/profile'.
 - Configures high-ratio Zstandard compression (zstd -Xcompression-level 19 -b 1M).
 - Generates hybrid UEFI and legacy BIOS boot sectors via xorriso.
2. Automate post-build verification inside 'build.sh':
 - Verifies the created ISO file at 'output/doomos-plasma-x86_64.iso'.
 - Generates SHA256 checksum in 'output/doomos-plasma-x86_64.iso.sha256'.
 - Verifies that file size is within the expected production range (2.8 GB - 3.4 GB).
3. Test-run the host build script ('./run-builder.sh') and monitor container compilation logs to

Phase 7: Virtual Testing & USB Deployment (Hardened for Mac & PC)

Key Hardening: Accounts for Apple Silicon host vs. x86_64 guest by providing adaptive QEMU parameters (TCG software emulation on Mac / KVM on Linux) and safe USB flashing.

Prompt 7

Act as a QA Systems Engineer on DoomOS.

Task: Create the cross-platform virtual machine testing script and bare-metal USB flashing util

Requirements:

1. Create 'test-qemu.sh' with adaptive host detection:
 - Detects host OS and CPU architecture (macOS Darwin vs Linux; arm64 vs x86_64).
 - On Linux x86_64: Runs QEMU with '-enable-kvm -cpu host'.
 - On macOS Apple Silicon: Runs QEMU with '-accel tcg,thread=multi -cpu max' or provides inst
 - Configures 4GB RAM, 4 CPU cores, OVMF UEFI firmware, and virtio graphics.
2. Create 'flash-usb.sh' with interactive disk safety checks:
 - Lists attached block devices ('lsblk' on Linux, 'diskutil list' on macOS).
 - Prompts for explicit device confirmation (e.g. /dev/sdb or /dev/diskN).
 - Refuses to write to system/internal drives.
 - Executes 'dd' with oflag=sync and status=progress.
3. Provide a step-by-step verification checklist confirming:
 - Live session SDDM autologin.
 - Wayland screen sharing and PipeWire audio.
 - Calamares Btrfs installation and Snapper snapshot generation.